

DermaIntegrate: 皮肤病多源数据集成与智能辅助诊断平台

许可协议: MIT License

运行环境: Python 3.10+ / Java 17+ / Node 18+

协作范式: 契约驱动开发, Kiko (智能管道) 与 Sun (应用平台) 领域正交、高度并行

摘要

本项目面向皮肤病专科门诊, 旨在解决传统医疗信息系统与AI推理系统紧耦合导致的计算阻塞、前端重度渲染DICOM导致的交互延迟, 以及纯大模型推理存在的不可解释性与幻觉风险。平台基于领域驱动设计(DDD), 采用视觉双轨防幻觉策略(自训练CNN特征锚定 + VLM语义推理), 对接FHIR R4医疗互联互通标准, 通过Multi-Agent架构提供皮肤镜影像与临床特征的综合诊断建议。系统从物理层面隔离“智能管道域”与“应用平台域”, 构建高可用、可解释、AI与工程深度解耦的医疗数字化范例。

问题定义与方法映射

现有痛点	本平台解决方案
纯VLM大模型诊断存在不可解释性与概率幻觉风险	视觉双轨约束: 自训练CNN生成Grad-CAM热力图作视觉锚点, 强制VLM语义与之对齐, 以传统CV的确定性约束生成式模型的概率性
前端强依赖Cornerstone.js解析DICOM导致渲染性能瓶颈	影像渲染优化: 基于皮肤镜影像无需调窗的光学特性, 后端离线转PNG+前端轻量滑动对比组件, 实现合理工程权衡
AI长连接耗尽业务线程池导致微服务雪崩	异步解耦防雪崩: REST+SSE异步契约解耦, Java WebClient反向代理流, 断连即终止GPU推理防算力泄漏
硬编码Mock数据导致系统缺乏可扩展性	标准化数据仿真: 引入HAPI FHIR构建Mock Server, 验证协议适配架构而非临时数据

系统架构设计

系统遵循“智能与业务解耦”原则, 严格划分为两大核心正交域: Kiko负责AI与数据治理闭环, Sun负责业务高可用与前端交互闭环。

④ 应用平台层 (Sun - React 18 + AntD + ECharts)		
医生工作站	影像滑动对比组件(原图/热力图)	AI诊断对话框
③ 应用服务层 (Sun - Java Spring Boot)		
患者360 API	FHIR Adapter	AI诊断异步代理 Sentinel熔断降级
② 智能管道层 (Kiko) ② 数据持久层 (Sun - DBA核心)		

Python FastAPI	PostgreSQL (混合建模: 关系+JSONB+GIN)
自训练CNN+VLM双轨	Redis (布隆过滤器+随机TTL防雪崩)
Multi-Agent+RAG	
① 数据接入与预处理层 (Kiko负责影像, Sun负责临床)	
DICOM离线转PNG/热力图叠加	HAPI FHIR Mock Server (仿真HIS/EMR)
② 基础设施层 (Kiko定拓扑, Sun主实施)	
Docker Compose	Nginx网关 华为云服务器部署 GPU显存隔离

关键工程实现与机制

架构设计中的防御性约束在系统中均具有明确的代码级实现映射, 以保障系统的鲁棒性。

1. 视觉双轨约束与可解释性机制

摒弃不可控的纯大模型输出, 采用“特征锚定+语义推理”双轨策略:

- DermAgent (提议):** 自训练ResNet50提取特征并生成Grad-CAM热力图 (提供确定性视觉锚点); 调用GLM-4V输出客观语义描述 (提供泛化能力)。
- IntegratorAgent (裁决):** 执行RAG检索, 拼接多源特征组装Prompt。
- 防幻觉强制拦截:** IntegratorAgent在代码层必须校验VLM描述与CNN高亮区域 (如“病灶边缘不规则”) 的对应关系, 以及Citation字段非空。若对齐失败, 抛出 `DiagnosisUncertainException`, 拒绝展示结果。

2. 异步流式交互与算力防泄漏机制

- Sun侧 (WebClient异步代理):** 严禁使用同步阻塞调用Kiko。Spring Boot通过 `webClient` 转发前端请求, 将Kiko的SSE流反向代理给前端, 防止Tomcat线程池被长连接耗尽。
- 断连感知防线:** 前端断开连接时, Sun的WebClient感知并中断对Kiko的SSE拉取; Kiko的FastAPI通过 `request.is_disconnected()` 感知, 立即终止后续GPU推理循环, 防止算力泄漏。

```
# Kiko侧 FastAPI 核心防泄漏逻辑
async def stream_diagnosis(request: Request, task_id: str):
    for step in agent_pipeline:
        if await request.is_disconnected():
            logger.warning(f"Client disconnected. Aborting GPU task: {task_id}")
            break # 终止推理, 释放显存
        yield sse_format(step)
```

3. 医学影像渲染优化与内存管控

针对前端医学影像交互的性能瓶颈, 执行严格的渲染隔离:

- 后端离线转码:** 针对皮肤镜影像无需调窗的光学特性, Kiko利用 `pydicom` 离线解析DICOM直接转存Web友好的PNG, 前端不引入Cornerstone等重度解析库。
- 前端内存防线:** 大图加载使用 `createObjectURL`, 在React组件卸载或切换患者时, 严格调用 `revokeObjectURL` 释放浏览器内存, 防止影像堆积导致的页面崩溃。

4. 多模态数据混合建模策略

针对AI特征的高效查询需求，采取关系+JSONB混合建模。除 `ai_features(JSONB)` 存储VLM描述等非结构化数据外，强制提取高频查询的 `lesion_type_varchar` 和 `confidence_numeric` 作为独立关系化列，并辅以 `GIN (ai_features jsonb_path_ops)` 索引，避免JSONB滥用导致的全表扫描性能衰退。

协作范式与项目拓朴

本项目由Kiko与Sun合作完成，采用契约驱动，领域严格正交。Kiko聚焦AI工程与数据治理，Sun聚焦全栈高可用与医疗标准落地。

角色矩阵

负责人	核心职责域	工程侧重
Kiko	智能管道（AI子系统、EMPI、影像预处理、RAG）	AI工程：双轨约束防幻觉、可解释性锚点、数据治理
Sun	应用平台（业务服务、前端交互、DBA、FHIR接入）	应用架构：WebClient异步、高可用架构、复杂交互与内存管控

目录结构

DermaIntegrate/	
├─ backend-ai/	# [Kiko全权负责] 智能管道层
│ ├─ agents/	# Multi-Agent交互 (DermAgent, IntegratorAgent)
│ ├─ cnn/	# ResNet50微调与Grad-CAM热力图生成
│ ├─ rag/	# LlamaIndex + Qdrant 检索增强生成
│ ├─ api/	# FastAPI路由 (SSE流式接口)
│ └─ preprocessing/	# DICOM离线转PNG与EMPI规则树匹配
├─ backend-app/	# [Sun全权负责] 应用服务层
│ ├─ fhir/	# HAPI FHIR Adapter解析标准R4资源
│ ├─ service/	# 患者360视图聚合、AI诊断异步代理
│ ├─ config/	# Sentinel熔断降级配置
│ └─ repository/	# PostgreSQL/Redis 混合查询与防穿透雪崩
├─ frontend/	# [Sun全权负责] 应用平台层
│ ├─ components/	# 影像滑动对比组件(原图/热力图)
│ ├─ hooks/	# useSSE状态机与断线重试
│ └─ views/	# 医生工作站与AI诊断对话框
├─ infra/	# [Kiko定拓朴, Sun主实施]
│ ├─ docker-compose.yml	# 异构微服务编排与GPU显存隔离
│ ├─ nginx/	# 网关路由与SSE超时配置
│ └─ mock-fhir-data/	# 虚拟患者JSON资源池
└─ docs/	# 契约文档与API定义
└─ api-contract.yaml	# 前后端与AI管道的SSE交互契约

部署与运行

1. 环境配置

系统采用Docker Compose编排异构微服务拓扑，屏蔽环境差异。

前提条件：如需体验AI辅助诊断功能，宿主机需安装 [NVIDIA Container Toolkit](#) 以支持GPU显存隔离。

```
git clone https://github.com/yourname/DermaIntegrate.git
cd DermaIntegrate

# 构建并启动所有服务
docker-compose up -d
```

► 核心依赖与版本控制

```
# Kiko侧 Python核心
torch==2.1.0+cu118
pytorch-gradiant==1.5.0
fastapi==0.109.0
llama-index==0.10.0
pydicom==2.4.3

# Sun侧 Java核心
spring-boot-starter-webflux # WebClient异步非阻塞
spring-boot-starter-data-redis
hapi-fhir-base==6.8.0      # FHIR R4协议解析
sentinel-datasource-nacos  # 熔断降级

# 基础设施
postgres:15                # 关系+JSONB+GIN
qdrant/qdrant:latest        # RAG向量库
redis:7                     # 布隆防穿透
```

2. 参数配置

在 `backend-ai/.env` 中配置视觉语言大模型接口：

```
# 视觉语言模型：GLM-4V（兼顾视觉理解与结构化输出）
VLM_API_KEY=xxxxxxxxxxxxxxxxxxx
VLM_BASE_URL=https://open.bigmodel.cn/api/paas/v4
VLM_MODEL=glm-4v
```

3. 运行模式

全链路联合启动：

```
docker-compose up -d
```

启动后访问 `http://localhost/` 即可打开医生工作站。系统内置HAPI FHIR Mock Server自动灌入虚拟患者数据，支持从“临床数据接入 -> EMPI匹配 -> 影像调阅 -> AI双轨流式诊断 -> 医生反馈”的全流程验证。

AI管道独立调试 (Kiko开发域)：

```
cd backend-ai
uvicorn api.main:app --reload --port 8000
# 仅启动FastAPI，可使用Mock影像路径直接测试SSE流式输出与防幻觉拦截
```

应用服务独立调试 (Sun开发域):

```
cd backend-app
mvn spring-boot:run
# 连接云端/本地AI管道，测试WebClient异步代理与Sentinel熔断
```

架构约束与设计原则

为保障医疗系统的鲁棒性、可解释性与高可用，代码级实施了以下硬性约束，任何联调与迭代均不可逾越：

- 可解释性约束**：严禁纯大模型黑盒输出。AI诊断结果若未生成Grad-CAM热力图与RAG Citation，视为推理失败，Kiko侧抛出异常，Sun侧触发降级提示，禁止向医生展示无依据的诊断结论。
- 渲染隔离约束**：前端严禁引入Cornerstone等重度DICOM库解析影像。所有医疗影像必须由Kiko在后端预处理为Web标准格式（PNG），前端仅负责图片渲染与内存管控。
- 阻塞降级约束**：严禁Sun以同步阻塞方式调用Kiko的AI接口。必须遵循异步+SSE契约，且必须实现断连中断GPU推理的机制，防止算力泄漏与Tomcat雪崩。
- 数据建模约束**：PostgreSQL中严禁将高频查询字段（如病灶类型、置信度）完全沉入JSONB。必须提取为独立关系化列并建B-Tree索引，JSONB仅用于存储VLM描述等非结构化特征。
- 契约先行约束**：Kiko与Sun的协作严格遵循 `docs/api-contract.yaml` 定义的数据结构。Kiko不可擅自变更SSE Event格式，Sun不可硬编码解析Kiko未定义的字段，确保异构系统高效并行开发。